



University of
Sheffield

COM1001 SPRING SEMESTER

Professor Phil McMinn

p.mcminn@sheffield.ac.uk

Data Confidentiality

What is “Data Confidentiality”?

**Confidentiality means
ensuring data is only accessible
to those authorised to view it.**

Examples of sensitive data you might want kept confidential include **passwords, addresses, phone numbers, and bank account numbers.**

Why Must We Take It Seriously?

As the maintainer of a web service, it's important that you keep sensitive user data confidential

If you suffer a confidentiality breach, and did not follow proper practices, you may be liable for damages!

<https://haveibeenpwned.com>

How Can We Keep Data Confidential?

First line:

- Sanitise external inputs
- Prevent unauthorised database acces, e.g. via SQL injection (Sequel handles for us)

Second line:

- **Encrypt sensitive data**, making it unreadable without the relevant information needed to decode it.

Passwords

Passwords should never be stored as plain text — **if your database is leaked, all passwords are exposed!**

`bcrypt` is a **hashing algorithm** designed specifically for passwords: it's **intentionally slow** (fast enough that real users don't notice, but slow enough to make brute-force attacks on a stolen database impractical!)

A **hash** is a one-way transformation — you can't reverse it to recover the original password.

```
irb(main):002> BCrypt::Password.create("test_password")  
=> "$2a$12$F5cp14WCcgWjjJrDAZLFEehGamfnnDRgS3t9U91aq4cmoyL.ijWgi"  
  
irb(main):003> BCrypt::Password.create("test_password")  
=> "$2a$12$PwrBz5k1M7dzRHW6/K.MU.v6a4AqMQFMi4UggBMaQ80lf9Eywka2."  
  
irb(main):004> BCrypt::Password.create("test_password")  
=> "$2a$12$Pih7cBsg0D0HRhMaHE1.M.4yxRvTu.XH5sxWb84xb1UrfngM2J35y"
```

Note how a **different hash** is created each time.

Why would it be a **bad** to create the same hash?

Salting

When bcrypt creates a hash, it uses some **randomly generated data (called a salt)** to generate it.

This means a new hash is generated each time.

The salt is bundled into the hash. So the final hash looks something like:

`$2a$12$someRandomSaltHere.andThenTheHashedPasswordHere`

Authentication

To authenticate users we get their string password entry (their password attempt) and use bcrypt to compare it against the one in the database:

```
BCrypt::Password.new(password_hash) == password_attempt
```

More is going on here than meets the eye (because the “`==`” operator is overloaded):

- `BCrypt::Password.new(password_hash)` extracts the salt from the stored password hash
- It uses the salt to produce a hash of the `password_attempt`
- It then compares the two hashes to see if they are identical.
- **There's no mathematical way to get the original password back.**
 - The only way is to authenticate is to try a password, hash it, and compare the result
 - ... and that's how it ensures everything is kept secure.

User Data

As well as passwords, we **should not store sensitive data in plain text** in the database either!

But we cannot use hashing — why?

Hashing v Encryption

Hashing is one-way, encryption is two-way.

Some data we need to be able to read back.

So we use encryption, rather than hashing.

Advanced Encryption Standard (AES)

AES-256-GCM is a widely trusted encryption algorithm used across the industry.

It uses a master key – a random key stored in a file on the server.

This is the secret that protects all encrypted data – it must be kept safe and **never committed to version control**.

Even if an attacker steals the database, the data is unreadable without the master key – they would need both to do any damage.

GCM – Practicals

Data is encrypted using a random “**Initialisation Vector**” (IV) – which performs a similar role to the salt used in hashing.

GCM also uses an authentication “**tag**”, which is designed to detect if stored data has been tampered with.

- **The tag works like a checksum.** A checksum works to summarise some data (e.g., 10, 20, 30 – checksum = 60). If one of the values is tampered with (e.g., 20 -> 21), the checksum is incorrect.
- **If the data has been tampered with, decryption returns an error rather than the tampered-with information.**

Password-Based Key Derivation

Function 2 (PBKDF2)

Converts a user's password into a key for AES.

AES requires a fixed length key (e.g. 16 bytes), so using a password directly is not usually possible.

Requires a salt – a random bit string. **This must be different for every user.** PBKDF2 combines the password and salt to ensure that users with the same password get different keys (mitigating against rainbow table attacks).

PBKDF2 does not make a password harder to guess. It should not be considered a substitute for a strong password.



Live Demonstration:

`data_confidentiality_example`

(from the code-examples directory of the GitHub repository)

Featuring:

- The database structure
- Demonstrating sign-up – and the encryption in the database
- Logging in following sign-up

Procedure for Encrypting User Data

helpers/security.rb

```
def self.encrypt(plaintext)
  return { data: nil, iv: nil, tag: nil } if plaintext.nil?
  cipher = OpenSSL::Cipher.new("AES-256-GCM")
  cipher.encrypt
  cipher.key = master_key
  iv = cipher.random_iv
  cipher.iv = iv
  data = cipher.update(plaintext) + cipher.final
  tag = cipher.auth_tag
  { data: data, iv: iv, tag: tag }
end
```


Procedure for Decrypting User Data

helpers/security.rb

```
def self.decrypt(encrypted_data, iv, tag)
  return nil if encrypted_data.nil? || iv.nil? || tag.nil?
  cipher = OpenSSL::Cipher.new("AES-256-GCM")
  cipher.decrypt
  cipher.key = master_key
  cipher.iv = iv
  cipher.auth_tag = tag
  cipher.update(encrypted_data) + cipher.final
end
```